Legal-IQ AI Integration Architecture

Here is a detailed breakdown of how the AI is integrated into **Legal-IQ**, what each function does, and how the entire system works together.

The AI system inside the `AI/` directory acts as the crucial "intelligence" layer. It uses Google's `gemini-2.5-flash` model for reasoning and `gemini-embedding-001` for vectorizing legal documents. The architecture strongly relies on a **RAG (Retrieval-Augmented Generation)** pattern where case text is transformed into vector embeddings and saved into your PostgreSQL database structure.

Here is the complete breakdown of every file, function, and responsibility:

1. The Core AI Engines (`AI/engine/`)

The core logic is divided into three main files: **Assistant**, **Ingestion**, and **Insights**.

A. AI Chat Assistant (`assistant.ts`)

This file powers the chat interface where lawyers or users can ask legal questions.

- **Function:** `getAssistantResponse(queryText: string, caseId?: string)`
  - **What it does:** It takes a user's question (`queryText`) and optionally a specific `caseId` .
  - **How it works:**
    1. If a `caseId` is provided, it connects to PostgreSQL and retrieves the exact context of the case (Title, Category, Court, Stage, Description).
    2. It bundles this specific case context with the user's question.
    3. It applies the `LEGAL_SYSTEM_PROMPT` (more on this below) to force the Gemini model into the persona of an expert legal researcher.
    4. It calls the `gemini-2.5-flash` model and returns the generated, analytical legal response.

B. Document Processing & Knowledge Base Builder (`ingestion.ts`)

This is the workhorse for building your local legal "Knowledge Base". When a lawyer uploads a legal PDF or text file for a case, it gets processed here.

- **Function:** `processDocument(filePath, caseId, documentId)`
  - **What it does:** Extracts text from `.pdf` , `.txt` , or `.md` files and prepares them for AI searches.
  - **How it works:**
    1. **Extraction:** It reads the raw text from the file (using `pdf-parse` for PDFs).
    2. **Chunking (`chunkText`):** Legal documents are massive, so it chops the text into smaller windows of **1200 characters** with a **200-character overlap** (to ensure no context is lost between cuts).
    3. **Embedding (`generateEmbedding`):** It runs each chunk through the `gemini-embedding-001` model. This model turns text into a large array of numbers (a vector) that mathematically represents the "meaning" of the text.
    4. **Storage (`storeEmbedding`):** It saves the original text chunk and its number vector into your Postgres database (`case_embeddings` table) linked to the `caseId` .

C. Automated Case Insights (`insights.ts`)

This is the most advanced feature. It automatically analyzes a case and provides a tactical strategy using the knowledge base processed by the ingestion engine.

- **Function:** `getCaseInsights(caseId: string)`
  - **What it does:** Automatically generates a comprehensive JSON report acting as a legal brief/strategy for a given case.
  - **How it works:**
    1. **Fetch:** Pulls the current case details from the database.
    2. **Query Vectorization:** It creates a search query based on the case's Title, Category, and Description, and turns it into a vector embedding.
    3. **Vector Search (`cosineSimilarity`):** It compares the search vector mathematically against all the document vectors stored in the database. It finds the **top 5 most relevant document chunks** (precedents) to act as legal context.
    4. **Generation:** It gives the case details and the top 5 matching precedents to the Gemini model using the `INSIGHTS_GENERATOR_PROMPT` .
    5. **Output:** Gemini returns a strictly structured JSON response containing tactical insights, evidentiary risks, precedent analysis, procedural next steps, and a probability assessment.

2. Personality & Structure (`AI/prompts/index.ts`)

LLMs behave best when given strict rules. This file defines exactly how the AI should act and format data.

- **`LEGAL_SYSTEM_PROMPT` :** This is the core instruction set given to the AI on every chat interaction. It tells the AI:
  - It is a Juris Doctor level legal researcher.
  - It must use precise legal terminology (e.g., "prima facie", "estoppel").
  - It must strictly prioritize the facts of the provided case.
  - It must explicitly warn that its output is for professional assistance, not final legal advice.
- **`INSIGHTS_GENERATOR_PROMPT` :** A dynamic template used by the `insights.ts` engine. It injects the case variables and local database context into the prompt, explicitly instructing Gemini to return a parseable **JSON object** with the 5 specific tactical keys required by the frontend UI.

3. Database Connection (`AI/utils/db.ts`)

- **What it does:** It creates a direct connection pool to your PostgreSQL database using the `pg` library.
- **How it works:** It reaches out to the `.env` file located in the `Backend/` folder to ensure it uses the exact same `DB_HOST` , `DB_NAME` , `DB_PASSWORD` , etc., as your main Node.js backend server. It exposes a fast `query()` wrapper to let the AI engines fetch case data and store embeddings independently.

In Summary: The Data Flow

1. **Upload:** User uploads a file -> `ingestion.ts` chunks it and saves it as vectors in Postgres.
2. **Chat:** User chats -> `assistant.ts` fetches the case state from Postgres, gives it to Gemini, and returns actionable advice.
3. **Strategy:** User asks for insights -> `insights.ts` grabs the case details, searches Postgres for locally uploaded vector precedents, and returns a fully structured JSON legal brief.